

# RELATÓRIO DE AUDITORIA DE SEGURANÇA

## DO.ing Club (doug-club)

Data da auditoria: 31 de agosto de 2026

---

### Escopo auditado

Código-fonte completo do repositório doug-club: rotas HTTP (routes/\*\*), controllers (app/Http/Controllers/\*\*), componentes Livewire (app/Livewire/\*\*), models Eloquent (app/Models/\*\*), recursos administrativos Filament (app/Filament/\*\*), configuração (config/\*\*, .env.example), seeders (database/seeders/\*\*) e views Blade (resources/views/\*\*). Não inclui pentest dinâmico nem dependências de terceiros (composer.lock/package-lock.json) linha a linha.

### Stack detectada

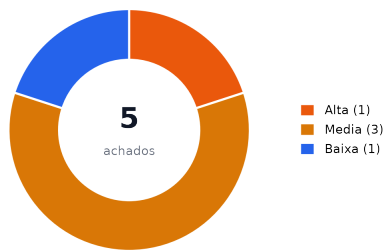
Laravel 13 (PHP 8.3) + Livewire 3 / Volt + Filament 4 (painel admin) + SQLite/MySQL via Eloquent. Sem Docker/CI/Helm/Terraform no repositório (deploy manual via Supervisor, ver docs/deploy).

## Nota metodológica — mapeamento das 5 categorias para esta stack

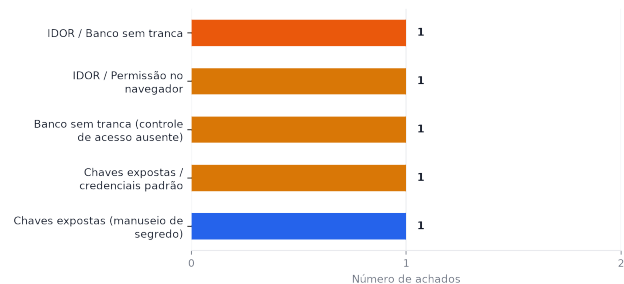
|                                        |                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Isolamento de dono/tenant</b>       | Não ha multi-tenant nem RLS (Postgres/Supabase). O isolamento é feito por coluna 'tier' do usuário (start/club/mentor) + coluna de posse (member_id/user_id/mentor_id) checada manualmente em cada Livewire component/controller, mais dois middlewares custom: EnsureAccessIsActive (assinatura ativa) e EnsureTier (tier mínimo por rota). |
| <b>Permissão definida no navegador</b> | Gates de UI (botões/links condicionais em Blade) foram cruzados com a checagem correspondente no componente Livewire ou controller server-side.                                                                                                                                                                                              |
| <b>IDOR</b>                            | Percorridos TODOS os controllers HTTP (app/Http/Controllers/**) e TODOS os componentes Livewire (app/Livewire/**) que recebem ID via route-model-binding, wire:click ou query string.                                                                                                                                                        |
| <b>Chaves expostas</b>                 | Buscado em config/**, .env.example, seeders, docs/deploy, routes e histórico de rotas por segredos hardcoded, defaults inseguros (env('X','secret')) e credenciais de seed.                                                                                                                                                                  |
| <b>XSS / entrada sem tratamento</b>    | Buscado por {!! !!} em Blade, innerHTML/dangerouslySetInnerHTML/v-html, eval/new Function, e renderização de Markdown/HTML de conteúdo de usuário em views e notificações.                                                                                                                                                                   |

## Resumo executivo

5 achados no total (1 alta, 3 media, 1 baixa) — 12 pontos fortes verificados.



Achados por severidade



Achados por categoria (cor = pior severidade na categoria)

## Pontos fortes (verificados no código)

- ✓ `app/Http/Controllers/Membros/VaultDocumentOpenController.php:15`  
Verifica ``$document->member_id === request()->user()->id`` antes de servir qualquer documento do cofre — bloqueia IDOR.
- ✓ `app/Http/Controllers/Membros/LessonMaterialDownloadController.php:14`  
Verifica ``$material->lesson->isAvailableFor($user)`` antes do download — padrão correto que faltou no controller de frameworks (achado #1).
- ✓ `app/Http/Middleware/EnsureTier.php` + `app/Models/User.php:72-79`  
Preview de persona do admin (``viewingTier()``) e explicitamente separado do ``tier`` real usado no gate de rota — o middleware nunca confia no valor de preview.
- ✓ `app/Http/Controllers/Membros/PreviewPersonaController.php:16`  
Exige ``is_admin`` no servidor antes de permitir a troca de persona de visualizacao — não é apenas escondido na UI.
- ✓ `routes/web.php` (grupo `membros/*`)  
Todas as rotas autenticadas aplicam consistentemente `['auth','verified','active']`, com `'tier:X'` adicional onde o conteúdo é restrito por plano.
- ✓ Toda a base de código (`app/**`)  
Nenhum uso de `DB::raw/whereRaw/selectRaw` com dados de entrada — 100% Eloquent com parameter binding, sem superfície de SQL injection encontrada.
- ✓ `resources/views/**/*.blade.php`  
Nenhuma ocorrência de ``{!! !!``, `v-html`, `dangerouslySetInnerHTML` ou renderização de Markdown sobre conteúdo de usuário (notas de mentor, bio, tags) — tudo passa por ``{{ }}`` escapado.
- ✓ `resources/js/*.js`  
Nenhum uso de `innerHTML/insertAdjacentHTML/document.write` com dado dinâmico nos componentes Alpine (`vimeo-progress`, `lesson-watermark`).
- ✓ `app/Http/Controllers/Webhooks/AbacatePayWebhookController.php:39`  
Comparação de segredo do webhook usa `hash_equals()` (timing-safe) e falha fechado (`abort 403`) quando o segredo não está configurado, em vez de aceitar por omissão.
- ✓ `.gitignore` + `git history`  
``.env`` nunca foi commitado (confirmado via ``git ls-files`` e ``git log -p -- .env``); nenhuma dependência AGPL no `composer.json`.

- 
- ✓ Todos os Models (app/Models/\*\*)  
Mass assignment protegido: ``$fillable`` explícito em cada model, nenhum uso de ``$guarded = []`` ou ``Model::unguard()``.
- 
- ✓ app/Livewire/Membros/Disponibilidade.php:66-71 e Agenda.php:78-93  
Mutacoes (remover bloco de agenda, cancelar sessao) sempre filtram por `Auth::id()` do dono, mesmo padrão replicado nos demais componentes revisados.

## Pontos fracos (riscos centrais)

O maior risco é o controle de acesso por nível de conteúdo (tier) ser aplicado de forma inconsistente entre controllers/actions quase idênticos: dois dos cinco achados (#1 e #2) são bypasses de tier em fluxos que tem um irmão correto no mesmo arquivo/trait, o que sugere um padrão a reforçar com teste automatizado (ver Recomendações). Os demais achados são superfícies acessórias — rotas de prototipo públicas, credenciais de seed e transporte de segredo de webhook — de exploração mais indireta, mas fáceis de corrigir.

## Achados detalhados

| Sev.  | Arquivo:linha                                                                                                                                                                                      | Descrição                                                                                                                                    |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Alta  | app/Http/Controllers/Membros/FrameworkPdfDownloadController.php:13-32<br>routes/web.php:41-43<br>resources/views/components/framework-card.blade.php:8-22                                          | <b>#1 Download de PDF de framework ignora o tier da aula vinculada (IDOR / controle de acesso quebrado)</b><br>IDOR / Banco sem tranca       |
| Media | app/Livewire/Concerns/TracksLessonProgress.php:25-51<br>app/Actions/MarkLessonAsWatching.php:9-18<br>app/Actions/MarkLessonAsCompleted.php:9-15<br>app/Actions/UpdateLessonWatchedSeconds.php:9-21 | <b>#2 Ações de progresso de aula (watchLesson/updateProgress/markCompleted) não validam o tier da lição</b><br>IDOR / Permissão no navegador |
| Media | routes/web.php:100<br>routes/prototype.php:1-22<br>app/Http/Controllers/Prototype/PrototypeController.php:1-210                                                                                    | <b>#3 Rotas /prototype/* publicadas sem qualquer autenticação ou guarda de ambiente</b><br>Banco sem tranca (controle de acesso ausente)     |
| Media | database/seeder/DatabaseSeeder.php:21-33<br>database/seeder/DemoDataSeeder.php:45-87                                                                                                               | <b>#4 Credenciais de administrador hardcoded em seeders, sem guarda de ambiente</b><br>Chaves expostas / credenciais padrão                  |
| Baixa | app/Http/Controllers/Webhooks/AbacatePayWebhookController.php:37-41                                                                                                                                | <b>#5 Segredo do webhook AbacatePay trafega via query string</b><br>Chaves expostas (manuseio de segredo)                                    |

**Alta**

## #1 — Download de PDF de framework ignora o tier da aula vinculada (IDOR / controle de acesso quebrado)

**Categoria:** IDOR / Banco sem tranca

**Local(is):**

- app/Http/Controllers/Membros/FrameworkPdfDownloadController.php:13-32
- routes/web.php:41-43
- resources/views/components/framework-card.blade.php:8-22

```
public function __invoke(Framework $framework): StreamedResponse
{
    abort_unless(
        $framework->hasUploadedFile() && Storage::disk('public')->exists($framework->pdf_path
    ),
        404,
    );
    // não ha checagem de $framework->lesson->isAvailableFor($user)
    ...
    return Storage::disk('public')->download($framework->pdf_path, ...);
}
```

**Por que é explorável**

Frameworks podem estar vinculados a uma Lesson com tier 'club' (Lesson::isAvailableFor() só libera para quem tem hasClubAccess()). O controller irmão, LessonMaterialDownloadController, faz abort\_unless(\$material->lesson->isAvailableFor(\$user)) antes de servir o arquivo (linha 14) — mas FrameworkPdfDownloadController nunca faz essa checagem. A rota só exige ['auth','verified','active'], sem 'tier'. No Blade (framework-card.blade.php), o botão 'Ver aula' é ocultado quando a lição não está disponível para o tier do usuário (linha 25), mas o botão 'Baixar PDF' é renderizado incondicionalmente (linhas 8-12) sempre que há arquivo. Um usuário tier 'start' autenticado pode acessar diretamente /membros/frameworks/{id}/download para qualquer framework, inclusive os vinculados a aulas exclusivas do CLUB, apenas descobrindo/iterando o ID (sequencial, incremental via chave primária).

**Impacto**

Bypass do paywall de tier: conteúdo pago (PDF de framework exclusivo CLUB) é obtido por assinantes do plano Start sem upgrade. O próprio DemoDataSeeder cria esse cenário real: o framework '45' é vinculado a uma Lesson tier=club E tem pdf\_path real, exercitando o bug com dados de exemplo.

**Correção sugerida**

Adicionar, no início do \_\_invoke, `abort\_unless(!\$framework->lesson\_id || \$framework->lesson->isAvailableFor(request()->user()), 404);` — mesmo padrão já usado em LessonMaterialDownloadController. Cobrir com teste de feature verificando 403/404 para usuário 'start' baixando framework de lição 'club'.

**Media**

## #2 — Ações de progresso de aula (watchLesson/updateProgress/markCompleted) não validam o tier da lição

**Categoria:** IDOR / Permissão no navegador

**Local(is):**

- app/Livewire/Concerns/TracksLessonProgress.php:25-51
- app/Actions/MarkLessonAsWatching.php:9-18
- app/Actions/MarkLessonAsCompleted.php:9-15
- app/Actions/UpdateLessonWatchedSeconds.php:9-21

```
public function watchLesson(int $lessonId, MarkLessonAsWatching $action): void
{
    $action->handle(Auth::id(), $lessonId); // só verifica Lesson::findOrFail
    $this->featuredLessonId = $lessonId;
}

// mas no MESMO trait, submitNpsScore FAZ a checagem correta:
public function submitNpsScore(int $lessonId, int $score, ...): void
{
    $lesson = Lesson::query()->find($lessonId);
    if (! $lesson || ! $lesson->isAvailableFor(Auth::user())) { return; }
    ...
}
```

### Por que é explorável

Métodos públicos de componentes Livewire são invocáveis via requisição AJAX direta ao endpoint Livewire, independente do que a UI renderiza (wire:click e só açúcar sintático). watchLesson, updateProgress e markCompleted aceitam lessonId vindo do cliente e chamam Actions que gravam em LessonProgress sem checar Lesson::isAvailableFor(\$user) — diferente de submitNpsScore, no mesmo trait, que faz exatamente essa checagem antes de agir. MarkLessonAsWatching só confirma que a Lesson existe (findOrFail); MarkLessonAsCompleted e UpdateLessonWatchedSeconds não checam nada.

### Impacto

Um membro tier=start pode forjar uma chamada Livewire com o lessonId de uma aula exclusiva CLUB e criar/alterar seu próprio LessonProgress (status completed, watched\_seconds) para essa aula, mesmo sem ter acesso a ela. Isso corrompe sinais usados em Radar::engagedStartMembers (contagem de lições 'start' completadas) e no dashboard. Mitigado parcialmente: o player de vídeo (lesson-player.blade.php:3) e o título em destaque (aulas.blade.php:19) re-checkam isAvailableFor antes de exibir o embed\_url, então o vídeo em si não vaza por esse caminho — o problema é a integridade dos dados de progresso, não exposição direta de mídia.

### Correção sugerida

Replicar em watchLesson/updateProgress/markCompleted a mesma guarda usada em submitNpsScore: buscar a Lesson e checar isAvailableFor(\$user) antes de chamar a Action, retornando cedo (sem efeito) quando não disponível.

## Media

## #3 — Rotas /prototype/\* publicadas sem qualquer autenticação ou guarda de ambiente

**Categoria:** Banco sem tranca (controle de acesso ausente)

**Local(is):**

- routes/web.php:100
- routes/prototype.php:1-22
- app/Http/Controllers/Prototype/PrototypeController.php:1-210

```
// routes/web.php
require __DIR__.'/auth.php';
require __DIR__.'/prototype.php'; // nenhum middleware, nenhum guard de ambiente

// routes/prototype.php
Route::prefix('prototype')->name('prototype.')->group(function () {
    Route::get('home', [PrototypeController::class, 'home'])->name('home');
    ... // 13 rotas, todas públicas
});
```

### Por que é explorável

Nenhuma das 13 rotas de /prototype/\* passa por middleware de autenticação, e o grupo não está protegido por `app()->environment('local')` nem por variável de config. Qualquer visitante não autenticado, em qualquer ambiente (inclusive produção, se este código for implantado como está), acessa /prototype/home, /prototype/dossies, /prototype/radar etc.

### Impacto

Os dados são mockados (config/prototype.php), não registros reais de banco, então não há vazamento de PII de membros reais. Ainda assim, expõe publicamente o design de produto ainda não lançado, a metodologia de mentoria (estrutura de 'dossies', 'pontes', especificação dos planos Start/Club) e a navegação completa do app para qualquer pessoa na internet — informação de negócio sensível para um produto em construção, além de superfície de ataque desnecessária exposta sem necessidade.

### Correção sugerida

Envolver o require de routes/prototype.php (ou o grupo de rotas dentro dele) em `if (app()->environment('local')) { ... }`, ou aplicar um middleware dedicado (ex: 'auth' + 'is\_admin') caso o prototipo precise ficar acessível em staging para o time.



## Media

## #4 — Credenciais de administrador hardcoded em seeders, sem guarda de ambiente

**Categoria:** Chaves expostas / credenciais padrão

**Local(is):**

- database/seeders/DatabaseSeeder.php:21-33
- database/seeders/DemoDataSeeder.php:45-87

```
User::factory()->create([
    'name' => 'Admin User',
    'email' => 'admin@example.com',
    'password' => Hash::make('123456789'),
    'is_admin' => true,
    'tier' => 'mentor',
]);
```

**Por que é explorável**

DatabaseSeeder::run() cria incondicionalmente um usuário admin@example.com com is\_admin=true e senha fixa '123456789' (mesma senha reaproveitada em mais 5 usuários no DemoDataSeeder). Nenhum dos dois seeders verifica `app()->environment('local')` ou `app()->isProduction()` antes de rodar. Se `php artisan db:seed` (ou um passo de deploy que chame `migrate --seed`) for executado contra produção — por engano ou por um pipeline de deploy mal configurado — cria-se uma conta admin com credencial pública e conhecida (está no código-fonte).

**Impacto**

Caso executado em produção: acesso administrativo total ao painel Filament (User::canAccessPanel() retorna true para is\_admin) com credencial trivial e pública no repositório, permitindo leitura/escrita de todos os recursos administrativos (membros, sessões, documentos do cofre, aplicações ao CLUB).

**Correção sugerida**

Adicionar guarda no topo de DatabaseSeeder::run() e DemoDataSeeder::run():  
`abort\_if(app()->isProduction(), 403, 'Seeders de demo não podem rodar em produção.');

ou envolver as chamadas com `if (! app()->environment('production')) { ... }`. Alternativamente, gerar senha aleatória via `Str::password()` e nunca reusar a mesma senha em vários usuários, mesmo em seeds de demonstração.

**Baixa**

## #5 — Segredo do webhook AbacatePay trafega via query string

**Categoria:** Chaves expostas (manuseio de segredo)

**Local(is):**

- app/Http/Controllers/Webhooks/AbacatePayWebhookController.php:37-41

```
$expectedSecret = config('services.abacatepay.webhook_secret');  
if (! $expectedSecret || ! hash_equals($expectedSecret, (string) $request->query('webhookSecret')) {  
    abort(403);  
}
```

### Por que é explorável

A comparação em si é correta (hash\_equals, com o valor conhecido como primeiro argumento, e o endpoint falha fechado quando o secret não está configurado). O ponto fraco é o transporte: o segredo compartilhado vai na query string (?webhookSecret=...) em vez de um header assinado. Query strings tendem a ser gravadas em logs de acesso do servidor/proxy, em ferramentas de APM/monitoramento de erro que capturam a URL completa, e potencialmente em caches de CDN/proxy.

### Impacto

Não é explorável diretamente por um atacante externo sem acesso a esses logs/ferramentas — a checagem em si bloqueia requisições sem o segredo correto. O risco é o segredo vazar por um canal secundário (log agregado, dashboard de erros) e depois ser reaproveitado para forjar eventos de pagamento (ex: liberar acesso CLUB via evento 'checkout.completed' falso).

### Correção sugerida

Se o provedor (AbacatePay) suportar, migrar para verificação via header assinado (HMAC do corpo da requisição) em vez de query string. Caso o provedor só ofereça query string, garantir que o log de acesso do servidor/proxy não persista query strings para essa rota e que ferramentas de APM redijam parâmetros sensíveis antes de enviar telemetria.

## Recomendações priorizadas

| Prio. | Recomendação                                                                                                                                                          |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P1    | Corrigir o IDOR de download de framework (achado #1) — bypass de paywall já demonstrável com os dados do próprio seeder de demonstração.                              |
| P1    | Adicionar guarda de ambiente aos seeders (achado #4) antes do próximo deploy, para eliminar o risco de credencial admin pública em produção.                          |
| P2    | Fechar a lacuna de controle de acesso nas ações de progresso de aula (achado #2), para preservar a integridade dos sinais usados no Radar.                            |
| P2    | Restringir /prototype/* a ambiente local ou a admin autenticado (achado #3) antes de qualquer deploy público.                                                         |
| P3    | Avaliar migrar a autenticação do webhook AbacatePay para header assinado, ou ao menos configurar redaction de query string nos logs (achado #5).                      |
| P3    | Adicionar suíte de testes de regressão de autorização (feature tests) cobrindo cada combinação tier x rota protegida, para travar os achados #1 e #2 permanentemente. |

# Issues para o GitHub

Texto completo, pronto para copiar e colar, de uma issue por achado acionável. Achados triviais relacionados não foi necessário agrupar nesta rodada (cada achado tem causa e correção distintas).

## --- ISSUE 1 ---

```
# [Segurança] Download de PDF de framework ignora o tier da aula vinculada (IDOR / controle de acesso quebrado)
```

```
**Labels sugeridas:** `security`, `severidade:alta`
```

### ## Descrição do problema

Frameworks podem estar vinculados a uma Lesson com tier 'club' (Lesson::isAvailableFor() só libera para quem tem hasClubAccess()). O controller irmão, LessonMaterialDownloadController, faz abort\_unless(\$material->lesson->isAvailableFor(\$user)) antes de servir o arquivo (linha 14) – mas FrameworkPdfDownloadController nunca faz essa checagem. A rota só exige ['auth','verified','active'], sem 'tier'. No Blade (framework-card.blade.php), o botão 'Ver aula' é oculta do quando a lição não está disponível para o tier do usuário (linha 25), mas o botão 'Baixar PDF' é renderizado incondicionalmente (linhas 8-12) sempre que há arquivo. Um usuário tier 'start' autenticado pode acessar diretamente /membros/frameworks/{id}/download para qualquer framework, inclusive os vinculados a aulas exclusivas do CLUB, apenas descobrindo/iterando o ID (sequencial, incremental via chave primária).

### ## Evidência

```
- `app/Http/Controllers/Membros/FrameworkPdfDownloadController.php:13-32`
- `routes/web.php:41-43`
- `resources/views/components/framework-card.blade.php:8-22`
```

```
```php
```

```
public function __invoke(Framework $framework): StreamedResponse
{
    abort_unless(
        $framework->hasUploadedFile() && Storage::disk('public')->exists($framework->pdf_path
    ),
        404,
    );
    // não ha checagem de $framework->lesson->isAvailableFor($user)
    ...
    return Storage::disk('public')->download($framework->pdf_path, ...);
}
```
```

### ## Impacto

Bypass do paywall de tier: conteúdo pago (PDF de framework exclusivo CLUB) é obtido por assinantes antes do plano Start sem upgrade. O próprio DemoDataSeeder cria esse cenário real: o framework '4S' é vinculado a uma Lesson tier=club E tem pdf\_path real, exercitando o bug com dados de exemplo.

### ## Sugestão de correção

Adicionar, no início do \_\_invoke, `abort\_unless(!\$framework->lesson\_id || \$framework->lesson->isAvailableFor(request()->user()), 404);` – mesmo padrão já usado em LessonMaterialDownloadController. Cobrir com teste de feature verificando 403/404 para usuário 'start' baixando framework de lição 'club'.

### ## Critérios de aceite

```
- [ ] Usuário tier=start recebe 404 ao acessar membros/frameworks/{id}/download quando {id} pertence a uma lesson tier=club
- [ ] Usuário tier=club/mentor continua baixando normalmente
- [ ] Framework sem lesson_id (livre) continua acessível a qualquer tier autenticado
- [ ] Teste de feature cobrindo os três casos acima adicionado em tests/Feature
```

## --- FIM ISSUE 1 ---

## --- ISSUE 2 ---

# [Segurança] Ações de progresso de aula (watchLesson/updateProgress/markCompleted) não validam o tier da lição

**Labels sugeridas:** `security`, `severidade:media`

## ## Descrição do problema

Métodos públicos de componentes Livewire são invocáveis via requisição AJAX direta ao endpoint Livewire, independente do que a UI renderiza (wire:click e só açúcar sintático). watchLesson, updateProgress e markCompleted aceitam lessonId vindo do cliente e chamam Actions que gravam em LessonProgress sem checar Lesson::isAvailableFor(\$user) – diferente de submitNpsScore, no mesmo trait, que faz exatamente essa checagem antes de agir. MarkLessonAsWatching só confirma que a Lesson existe (findOrFail); MarkLessonAsCompleted e UpdateLessonWatchedSeconds não checam nada.

## ## Evidência

- `app/Livewire/Concerns/TracksLessonProgress.php:25-51`
- `app/Actions/MarkLessonAsWatching.php:9-18`
- `app/Actions/MarkLessonAsCompleted.php:9-15`
- `app/Actions/UpdateLessonWatchedSeconds.php:9-21`

```
```php
```

```
public function watchLesson(int $lessonId, MarkLessonAsWatching $action): void
{
    $action->handle(Auth::id(), $lessonId); // só verifica Lesson::findOrFail
    $this->featuredLessonId = $lessonId;
}
```

// mas no MESMO trait, submitNpsScore FAZ a checagem correta:

```
public function submitNpsScore(int $lessonId, int $score, ...): void
{
    $lesson = Lesson::query()->find($lessonId);
    if (! $lesson || ! $lesson->isAvailableFor(Auth::user())) { return; }
    ...
}
```

## ## Impacto

Um membro tier=start pode forjar uma chamada Livewire com o lessonId de uma aula exclusiva CLUB e criar/alterar seu próprio LessonProgress (status completed, watched\_seconds) para essa aula, mesmo sem ter acesso a ela. Isso corrompe sinais usados em Radar::engagedStartMembers (contagem de lições 'start' completadas) e no dashboard. Mitigado parcialmente: o player de vídeo (lesson-player.blade.php:3) e o título em destaque (aulas.blade.php:19) re-ancam isAvailableFor antes de exibir o embed\_url, então o vídeo em si não vaza por esse caminho – o problema é a integridade dos dados de progresso, não exposição direta de mídia.

## ## Sugestão de correção

Replicar em watchLesson/updateProgress/markCompleted a mesma guarda usada em submitNpsScore: buscar a Lesson e checar isAvailableFor(\$user) antes de chamar a Action, retornando cedo (sem efeito) quando não disponível.

## ## Critérios de aceite

- [ ] watchLesson/updateProgress/markCompleted não gravam LessonProgress quando a lesson não está disponível para o tier do usuário autenticado
- [ ] Comportamento existente preservado para lições disponíveis (start sempre, club/mentor para hasClubAccess())
- [ ] Teste de feature chamando os tres métodos via Livewire::test() com usuário start e lesson club, garantindo que LessonProgress não é criado

## --- FIM ISSUE 2 ---

## --- ISSUE 3 ---

```
# [Segurança] Rotas /prototype/* publicadas sem qualquer autenticação ou guarda de ambiente

**Labels sugeridas:** `security`, `severidade:media`

## Descrição do problema
Nenhuma das 13 rotas de /prototype/* passa por middleware de autenticação, e o grupo não está
protegido por `app()->environment('local')` nem por variável de config. Qualquer visitante n
ão autenticado, em qualquer ambiente (inclusive produção, se este código for implantado como
está), acessa /prototype/home, /prototype/dossies, /prototype/radar etc.

## Evidência
- `routes/web.php:100`
- `routes/prototype.php:1-22`
- `app/Http/Controllers/Prototype/PrototypeController.php:1-210`

```php
// routes/web.php
require __DIR__.'/auth.php';
require __DIR__.'/prototype.php'; // nenhum middleware, nenhum guard de ambiente

// routes/prototype.php
Route::prefix('prototype')->name('prototype.')->group(function () {
    Route::get('home', [PrototypeController::class, 'home'])->name('home');
    ... // 13 rotas, todas públicas
});
```

## Impacto
Os dados são mockados (config/prototype.php), não registros reais de banco, então não há vazamento de PII de membros reais. Ainda assim, expõe publicamente o design de produto ainda não lançado, a metodologia de mentoria (estrutura de 'dossies', 'pontes', precificação dos planos Start/Club) e a navegação completa do app para qualquer pessoa na internet – informação de negócio sensível para um produto em construção, além de superfície de ataque desnecessária exposta sem necessidade.

## Sugestão de correção
Envolver o require de routes/prototype.php (ou o grupo de rotas dentro dele) em `if (app()->environment('local')) { ... }`, ou aplicar um middleware dedicado (ex: 'auth' + 'is_admin') caso o prototipo precise ficar acessível em staging para o time.

## Critérios de aceite
- [ ] GET /prototype/* retorna 404 (rota inexistente) quando APP_ENV != local, ou passa a exigir autenticação+is_admin
- [ ] Nenhuma rota prototype.* aparece em `php artisan route:list` quando executado com APP_ENV=production
- [ ] Time confirma se staging precisa de acesso; se sim, adicionar middleware equivalente em vez de deixar público
```

## --- FIM ISSUE 3 ---

## --- ISSUE 4 ---

```
# [Segurança] Credenciais de administrador hardcoded em seeders, sem guarda de ambiente

**Labels sugeridas:** `security`, `severidade:media`

## Descrição do problema
DatabaseSeeder::run() cria incondicionalmente um usuário admin@example.com com is_admin=true e senha fixa '123456789' (mesma senha reaproveitada em mais 5 usuários no DemoDataSeeder). Nenhum dos dois seeders verifica `app()->environment('local')` ou `app()->isProduction()` antes de rodar. Se `php artisan db:seed` (ou um passo de deploy que chame `migrate --seed`) for executado contra produção – por engano ou por um pipeline de deploy mal configurado – cria-se uma conta admin com credencial pública e conhecida (está no código-fonte).

## Evidência
- `database/seeders/DatabaseSeeder.php:21-33`
- `database/seeders/DemoDataSeeder.php:45-87`

```php
User::factory()->create([
    'name' => 'Admin User',
    'email' => 'admin@example.com',
    'password' => Hash::make('123456789'),
    'is_admin' => true,
    'tier' => 'mentor',
]);
```

## Impacto
Caso executado em produção: acesso administrativo total ao painel Filament (User::canAccessPanel() retorna true para is_admin) com credencial trivial e pública no repositório, permitindo leitura/escrita de todos os recursos administrativos (membros, sessões, documentos do cofre, aplicações ao CLUB).

## Sugestão de correção
Adicionar guarda no topo de DatabaseSeeder::run() e DemoDataSeeder::run(): `abort_if(app()->isProduction(), 403, 'Seeders de demo não podem rodar em produção.'); ou envolver as chamadas com `if (! app()->environment('production')) { ... }`. Alternativamente, gerar senha aleatória via `Str::password()` e nunca reusar a mesma senha em vários usuários, mesmo em seeds de demonstração.

## Critérios de aceite
- [ ] DatabaseSeeder::run() e DemoDataSeeder::run() abortam (ou viram no-op) quando app()->environment('production')
- [ ] Documentado em docs/deploy que `db:seed` nunca deve rodar contra produção
- [ ] (Opcional) senha de demo deixa de ser fixa/reaproveitada entre usuários
```

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

```
# [Segurança] Segredo do webhook AbacatePay trafega via query string

**Labels sugeridas:** `security`, `severidade:baixa`

## Descrição do problema
A comparação em si é correta (hash_equals, com o valor conhecido como primeiro argumento, e o endpoint falha fechado quando o secret não está configurado). O ponto fraco é o transporte: o segredo compartilhado vai na query string (?webhookSecret=...) em vez de um header assinado. Query strings tendem a ser gravadas em logs de acesso do servidor/proxy, em ferramentas de APM/monitoramento de erro que capturam a URL completa, e potencialmente em caches de CDN/proxy.

## Evidência
- `app/Http/Controllers/Webhooks/AbacatePayWebhookController.php:37-41`

```php
$expectedSecret = config('services.abacatepay.webhook_secret');
if (! $expectedSecret || ! hash_equals($expectedSecret, (string) $request->query('webhookSecret')) {
    abort(403);
}
```

## Impacto
Não é explorável diretamente por um atacante externo sem acesso a esses logs/ferramentas – a checagem em si bloqueia requisições sem o segredo correto. O risco é o segredo vaziar por um canal secundário (log agregado, dashboard de erros) e depois ser reaproveitado para forjar eventos de pagamento (ex: liberar acesso CLUB via evento 'checkout.completed' falso).

## Sugestão de correção
Se o provedor (AbacatePay) suportar, migrar para verificação via header assinado (HMAC do corpo da requisição) em vez de query string. Caso o provedor só ofereça query string, garantir que o log de acesso do servidor/proxy não persista query strings para essa rota e que ferramentas de APM redijam parâmetros sensíveis antes de enviar telemetria.

## Critérios de aceite
- [ ] Confirmar com a documentação do AbacatePay se há alternativa de assinatura via header
- [ ] Se disponível, migrar a validação para o header assinado, mantendo query string apenas como fallback documentado
- [ ] Configurar redaction de query string para essa rota nas ferramentas de log/APM em uso
```

--- FIM ISSUE 5 ---